

Очередь запросов

Сервер приложения делится на несколько пулов. В типовом случае их два - основной и служебный. Один пул обслуживает некоторую часть потока запросов на сервере приложения.

У сервисов требования к дисциплине обслуживания поступающего потока запросов различаются. Для реализации этих требований используется интерфейс очереди обслуживания.

AbstractTaskQueue

Платформа позволяет кастомизировать логику очереди обслуживания, т.е. реализовать свою очередь.

Для этого необходимо реализовать интерфейс [sbis::rpc::AbstractTaskQueue](#), который включает в себя следующие методы:

- [Init](#)

Выполняет настройку очереди запросов в зависимости от того пула, для которого она будет применяться.

```
virtual void Init( AbstractExecutorPool& pool ) = 0;
```

- [Put](#)

Принимает запрос (объект типа [AbstractRequest](#)) в очередь и ничего не возвращает. Данный метод вызывается экземпляром класса [AbstractExecutorPool](#).

```
virtual void Put( UniquePtr< AbstractRequest >::type request ) = 0;
```

- [Get](#)

Просит у очереди выдать запрос на исполнение (очередь не обязана выдавать запросы, даже если они у нее есть).

```
virtual UniquePtr< AbstractRequest >::type Get() = 0;
```

- [Finish](#)

Уведомляет об окончании исполнения запроса, ранее выданного из очереди.

```
virtual void Finish( const AbstractRequest& request ) = 0;
```

- [Name](#)

Возвращает название очереди.

```
virtual String Name() const = 0;
```

На основе значения, возвращаемого методом выполняется сопоставление класса очереди со значением параметра [Ядро.Сервер приложений.Очередь](#).

При конфигурации сервиса мы вводим в этот параметр некоторое строковое значение и если мы находим в системе имплементацию интерфейса `AbstractTaskQueue`, у которого метод `Name` возвращает это значение, то создаем экземпляр данной очереди и используем ее для пула приложения.

Каждая имплементация должна быть зарегистрирована под именем в фабрике с помощью объекта [TaskQueueFactory::Registrar](#).

Основные стратегии обработки входящих запросов:

1. FIFO (First In, First Out) - стратегия, при которой на обработку первым подается запрос, который поступил в очередь раньше остальных. Является стратегией по умолчанию.
2. PQ - стратегия, при которой запрос на обработку выбирается исходя из приоритета запросов, находящихся в очереди. Приоритет рассчитывается исходя из текущей загрузки пула рабочих процессов. Эта очередь

реализуется отдельным модулем Load balancing в составе [SDK](#). Подробнее описано в статье [Затороустойчивый сервис \(Load balancing\)](#).

Примеры кастомных имплементаций

Если вышеуказанные стратегии не совсем подходят для конкретного сервиса, используются кастомные имплементации, примеры которых приведены ниже.

MWQ

Эта очередь реализована в Сервисе кэширования, у которого есть три класса методов:

- на чтение из кэша
- на инвалидацию кэша
- на запись в кэш.

Наибольший приоритет отдается запросам на чтение, второй приоритет - запросам на инвалидацию и самый низкий - запросам на запись.

Previewer

[Сервис Previewer](#) включен в большое количество публичных доменов. Соответственно, возникают ситуации, при которых либо из одного домена поступает очень много запросов, либо на сервис поступают запросы на одно и то же изображение. Обе ситуации приводят к сбою в работе сервиса и для его устранения реализована специальная очередь Previewer.

PSQueue

Такая очередь обеспечивает незаполнение пула рабочих потоков [Сервиса представления](#) запросами от одного пользователя.

За пределами Сервиса представления очередь PSQueue не может быть применена, т.к. эта имплементация интерфейса расположена в модуле Сервиса представления, соответственно, те, у кого нет этого модуля, воспользоваться ей не могут.

SESSION_BALANSED_QUEUE

Модуль Session Queue расположен в репозитории [engine](#) (не входит в состав SDK) и обеспечивает предотвращение завала сервиса запросами из-под одного аккаунта и пользователя.

SESSION_BALANSED_QUEUE при выборе из очереди отдает приоритет тем сессиям, которые меньше нагружают сервер. Например, на сервере очередь, а на одном из рабочих процессов завершился запрос. Очередь должна выбрать, какой запрос сейчас будет выполнен на освободившемся процессе. Для каждой сессии учитывается время выполнения запросов. На обработку выбирается запрос сессии, у которой это время меньше. Для учета берется "скользящее среднее" время, когда более ранние запросы вносят меньший вклад в среднее.

PURQ

Представляет собой очередь с ограничением по количеству воркеров, занимаемых одним клиентом. Соответственно, когда один клиент начинает генерировать большую нагрузку (типовой сценарий - первоначальная выгрузка из сторонней CRM системы), его запросы начинают копиться в очереди, никак не влияя на других клиентов, пользующихся сервисом. Поэтому при большой очереди от одного клиента никак не страдают другие.



Очередь работает только для основного пула!

Для включения очереди достаточно задать значение PURQ для параметра [Ядро.Сервер приложений.Очередь](#), а также настроить параметры:

- [Ядро.PerUserRestrictionQueue.IsBalancedByClient](#) - работа с идентификацией по пользователю или по клиенту. По умолчанию задана очередь по пользователю.
- [Ядро.PerUserRestrictionQueue.MaxTasksPerUser](#) - максимальное количество одновременно обрабатываемых запросов от одного пользователя. Значение 0 отключает данное ограничение.

При значении 0 очередь будет брать запросы каруселью по пользователям, т.е. берем один запрос из очереди одного пользователя, переходим к следующему пользователю и т.д.

- [Ядро.PerUserRestrictionQueue.UserQueryLimit](#) - максимальное количество запросов в очереди от одного пользователя. Значение 0 отключает данное ограничение. Если количество запросов в очереди, превысит заданное в параметре, остальные запросы будут отброшены.

Если параметр не задан, очередь будет безразмерной.

Эти параметры необходимо зафиксировать и после этого перезапустить сервис последовательно.